

Fenwick trees

druin

Aug 2025

Introduction

$$\begin{aligned} \text{BIT}[x] &= a[x], & \dots \text{if } x \text{ is odd} \\ &= a[1] + \dots + a[x], & \dots \text{if } x = 2^n \end{aligned}$$

- Remember the list is 1-indexed, not 0 indexed.
- To generalize this, every index in the array stores the cumulative sum from the index i to $i - (1 \ll r) + 1$ (both inclusive), where r represents the last set bit in the index.
- To get last bit do $x \& (-x)$; $-x = x + 1$ in signed integer representation.
- $\text{BIT}[6] = \text{BIT}[6] + \text{BIT}[5]$;
- To update, let's say $x = 13$, $\text{BIT}[x]$ has to be increased by 2, then $\text{BIT}[14]$ and $\text{BIT}[16]$ have to be increased as well, as they contain $\text{BIT}[13]$.
- $\text{BIT}[13] = \text{BIT}[1101] = \text{BIT}[1101] + d$,
- $\text{BIT}[1101 + x \& -x] = \text{BIT}[1101 + 0001 = 1110 = 14] = \text{BIT}[14] + d$,
- $\text{BIT}[1110 + x \& -x] = \text{BIT}[1110 + 0010] + d = \text{BIT}[10000] + d$

1 When to use Fenwick trees?

- Associative that is $f(f(a,b),c) = f(a, f(b,c))$
- Has an inverse.

For example:

- Addition has inverse subtraction (this example we have discussed)
- Multiplication has inverse division
- GCD() has no inverse, so we can't use BIT to calculate range gcd's
- Sum of matrices has inverse
- Product of matrices would have inverse if it is given that matrices are non-degenerate i.e. determinant of any matrix is not equal to 0

```

BIT.java
import java.util.*;

public class BIT{
    public static void main(String[] args) {
        int arr[] = {1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16};

        int tree[] = makeBIT(arr);
        for(int i=arr.length-1;i>=0;i--){
            System.out.printf("BIT[%d] = %d\n", i, tree[i]);
        }
        update(tree, 13, 2);
        for(int i=arr.length-1;i>=0;i--){
            System.out.printf("BIT[%d] = %d\n", i, tree[i]);
        }
    }
    public static int[] makeBIT(int []arr){
        int BITree[] = new int[arr.length];

        /*
        for(int i=1;i<arr.length;i+=2){
            BITree[i] = arr[i];
        }
        */
        /* No need to have the below loop(bad complexity for powers of 2), instead use t
        for(int i=0;i<arr.length;i++){
            int lastbit = i&-i;    // could have i-lastbit-1 here itself rather than co
            //System.out.printf("i=%d, lastbit=%d\n", i,lastbit);
            for(int j=i;j>=i-lastbit+1;j--){
                BITree[i] += arr[j];
            }
        }
        */
        for(int i=0;i<arr.length;i++){
            update(BITree, i, arr[i]);
        }
        return BITree;
    }
    static void update(int[] BITree, int index, int delta){
        for(int i=index; i<BITree.length;i += i&-i){
            BITree[i] += delta;
        }
    }
    static int querySumOfPrefix(int[] BITree, int index){
        int sum=0;

        for(int i=index; i>=0;i -= i&-i) {           // i&-i gives the last set bit
            sum += BITree[i];
        }
    }
}

```

```
    return sum;  
}  
}
```

